PARSER D'AUTOMATES EN GAL

SYNTAXE GAL

La syntaxe GAL est décrite dans le fichier SYNTAXE.md

LE PARSER

- Le parser est écrit en `JavaCC`, il est fourni par les enseignants sous la forme d'un fichier `parser.jj` dans le répertoire `src/gal/parser`.
- ⚠️ le répertoire contient un `.gitignore` qui demande d'ignorer les fichiers `.java` générer par `JavaCC`. Vous pouvez le supprimer puisque vous ne modifierez pas le parser.
- il produit un AST (*Abstract Syntaxe Tree*) qui correspond à l'arbre de dérivation construit lors du parsing.
- `TODO` À partir de l'AST vous devrez générer une représentation exécutable des automates

PRISE EN MAIN

Pour se familiariser avec le structure de l'AST que génère le parser, vous pouvez tester le parser en ligne de commande.

dans le répertoire `src/gal/demo/test/`

1. `apt get install javacc`
2. `make parser`

Pour chaque fichier `fichier.gal` vous pouvez ensuite faire

1. `make fichier.txt` pour vérifier que votre `fichier.gal` est accepté par le parser
2. `make fichier.ast.jpg` pour avoir une représentation graphique de l'AST généré par le parser
3. `make fichier.aut.jpg` pour avoir une représentation graphique de l'automate correspondant à `fichier.gal`

Le parser propose deux sorties à partir d'un [fichier .gal]

- avec l'option `-ast`: il produit, au format `.dot`, l'arbre de dérivation généré par le parser (voir exemples_ast.pdf généré par graphviz à partir du fichier `.dot`).

```
java gal.Parser -file exemples.gal -ast > exemples_ast.dot
```

- avec l'option `-aut`: il produit la représentation graphique de l'automate au format `.dot` (voir exemples_aut.pdf généré par graphviz à partir du fichier `.dot`).

```
java gal.Parser -file exemples.gal -aut > exemples_aut.dot
```

- Le format `.dot` peut être visualisé avec l'outil graphviz.

> Ces deux sorties vous montre, de manière graphique, la différence entre * l'Arbre Syntaxique (AST) construit par la parser et * l'automate qu'on a en tête et qu'il faut reconstruire à partir de l'AST.

UTILISATION DU PARSER DANS UN DÉVELOPPEMENT

Ci-dessous une utilisation possible du parser pour charger des automates à partir d'un fichier `.gal`

```
List<FSM> loadAutomata(String filename) {
    LinkedList<FSM> fsm_list;
    try {
      AST ast = (AST) Parser.from_file(filename);
      ...
      // TODO à vous de constuire les FSM (automates exécutables) à partir
de l'AST
      ...
      return FSM_list;
    } catch (Exception ex) {
      return null;
    }
  }
```

CONSTRUCTION DES AUTOMATES À PARTIR DE L'AST

Deux manières de construire les automates à partir de l'AST généré par le parser:

1. L'AST est un arbre

que vous pouvez parcourir en explorant ses champs pour y piocher les informations nécessaires à la construction des automates.

2. Le package AST est fourni avec un Visitor

La méthode `accept(Visitor)` de l'AST déclenche un parcours d'arbre et appelle les fonctions du Visitor sur chacun des noeuds de l'arbre.

Pour générer les automates il *suffit* donc de fournir l'implémentation de ces fonctions en respectant l'interface IVisitor.

Voici trois exemples d'implémentations de l'interface `IVisitor`

À partir de l'AST construit par le parser GAL,

- Ast2Gal.java génére la sortie `-src` au format `.txt`
- AstPrinter.java génère la sortie `-ast` au format `.dot`
- AutPrinter.java génère la sortie `-aut` au format `.dot`

```
AUTHOR: Michaël PÉRIN, Polytech'Grenoble, Univ. Grenoble Alpes
```